Ops-Free Amazon SES Engine & Dashboard Wrapper

Production-Grade Software Requirements Specification (SRS)

Version: 1.0

Status: Engineering Baseline

Date: 26 September 2026

Technology baseline: Next.js 15 App Router, React, Tailwind CSS, shadcn/ui, Supabase Auth, PostgreSQL, Drizzle ORM, Upstash Redis, BullMQ, AWS SDK v3, Vercel.

Document Purpose

This SRS defines the product, architecture, requirements, data model, API contracts, security controls, operational behavior, testing criteria, and deployment model for a managed developer platform wrapping Amazon Simple Email Service (SES). The platform provides a Resend/Postmark-style developer experience while retaining customer-owned SES as the delivery infrastructure.

The platform reduces SES operational overhead around domain verification, DNS, queueing, rate limiting, bounce/complaint handling, suppression, templates, analytics, credential management, and sandbox-exit preparation.

Engineering note: the product cannot guarantee AWS production approval, inbox placement, or immunity from AWS reputation/policy enforcement.

1. Executive Summary & Problem Statement

Raw Amazon SES is inexpensive and scalable, but its primitives expose operational responsibilities directly to application teams. Developers must understand SES identities and regions, configure SPF/DKIM/DMARC, manage sending quotas, handle sandbox restrictions, process bounces and complaints, implement suppression, build retry/backoff behavior, secure AWS credentials, and create their own developer-facing dashboard.

Core operational pain points

- **Manual SPF/DKIM/DMARC setup:** identity verification and DNS records are easy to misconfigure.
- **Bounce/complaint handling:** repeated sends to invalid or complaining recipients can create reputation risk.
- **Rate-limit failures:** bursts can trigger provider throttling such as `MaxSendRateExceeded`.
- **AWS Sandbox:** new SES accounts may be restricted to verified recipients and low quotas until AWS grants production access.
- **Credential complexity:** IAM access keys must never reach browsers, source repositories, or logs.
- **Developer-experience gap:** SES provides APIs and primitives rather than a complete transactional-email control plane.

Value proposition

Provide a Resend-like developer experience at SES infrastructure economics: one API, asynchronous queueing, managed domains, templates, suppression, analytics, and AWS operational assistance.

2. System Architecture & Data Flow

```
flowchart LR
  A[Client App / SDK] -->|HTTPS POST /api/v1/send| B[Next.js 15 API Route]
  B --> C[API Key Auth + Validation]
  C --> D[Redis / Upstash]
  D --> E[BullMQ Queue]
  E --> F[Worker / Queue Consumer]
  F --> G[AWS SDK v3]
  G --> H[Amazon SES]
  H -->|Delivery / Bounce / Complaint events| I[AWS SNS]
  I -->|HTTPS webhook| J[Next.js SNS Webhook]
  J --> K[Verify SNS signature + parse event]
```

```
K --> L[(PostgreSQL / Supabase)]
L --> M[Suppression List]
L --> N[Email Logs]
L --> O[Analytics Dashboard]
A --> O
O -->|Domain / Template / API Key management| B
```

Architecture principles

1. Separate the control plane from the delivery plane.
2. Treat API acceptance as durable queue acceptance, not delivery confirmation.
3. Keep AWS credentials strictly server-side.
4. Make workers idempotent.
5. Treat SES region as explicit because identities and quotas are region-scoped.
6. Make webhook processing authenticated and idempotent.
7. Use distributed rate limiting across workers.

3. Product Scope & Personas

Persona	Primary need
SaaS developer	Send transactional email without managing SES internals
Startup engineering team	Domains, templates, logs, suppressions and queues in one system
DevOps/platform engineer	Observe quotas, retries, queue health and failures
Product/admin	Search message activity and delivery outcomes

In scope

- Transactional email API
- Customer-owned SES credentials
- Staging and Production environments
- Domain/DNS verification
- API key management
- Visual templates
- Redis/BullMQ queueing
- Bounce/complaint ingestion
- Suppression
- Analytics and activity logs
- Sandbox-exit assistance
- Auditability and observability

Out of scope for v1

- Bulk marketing automation
- List acquisition
- Guaranteed inbox placement
- Independent mail-transfer infrastructure
- Automatic AWS account ownership changes
- Billing unless implemented as a separate module

4. Functional Requirements

FR-1 — Authentication & AWS Credential Vault

Users shall authenticate through Supabase Auth. Each user may maintain separate Staging and Production SES credentials.

Requirements:

- Encrypt secret access keys before database persistence.
- Never return secret keys after creation.
- Store AWS region explicitly.
- Validate credentials through a server-side AWS call.
- Support credential activation/revocation.
- Enforce tenant ownership on every credential operation.

FR-2 — Domain & DNS Verification Manager

The system shall provide a guided identity-verification checklist.

Requirements:

- Display SES verification state.
- Display expected SPF, DKIM and DMARC records.
- Poll verification state with bounded retries.
- Show last-check timestamp.
- Detect common DNS errors where observable.
- Keep environment and region explicit.

FR-3 — High-Throughput Sending API

`POST /api/v1/send` shall accept:

- API key
- sender
- recipient array
- subject
- HTML/text content
- template ID
- template variables
- tags/metadata
- optional idempotency key

The endpoint validates, checks suppression, persists a log and durably enqueues a job.

FR-4 — Rate-Limiting & Burst Queueing Engine

BullMQ shall manage jobs, concurrency, retries and backoff. A distributed limiter shall throttle outgoing SES calls according to the customer's configured SES sending rate.

The system must not intentionally exceed the configured provider rate.

FR-5 — Automatic Bounce & Complaint Webhooks

The SNS receiver shall:

1. Verify SNS authenticity.

2. Validate the expected topic.

3. Parse SES notification payloads.

4. Persist provider events.

5. Update email status.

6. Add hard-bounced/complaining addresses to suppression according to policy.

7. Block future sends to active suppressed addresses.

8. Process duplicate SNS messages idempotently.

FR-6 — Visual Email Template Builder

Support:

- Rich text / HTML editing
- Plain-text fallback
- `{{variable}}` interpolation
- Variable catalog
- Instant preview
- Template versions
- Controlled sample data
- Test-send workflow

FR-7 — Analytics & Activity Logs

Dashboard shall expose:

- Sent
- Delivered
- Bounced
- Complained
- Failed
- Suppressed
- Searchable logs
- Recipient/message ID lookup
- Error codes
- Provider request IDs
- Queue status
- Environment filtering

FR-8 — AWS Sandbox Exit Request Assistant

The assistant shall collect:

- product/use-case
- website
- expected daily and peak volume
- recipient acquisition/consent process
- bounce/complaint controls

- suppression process
- domain authentication
- requested AWS region and quota

It shall generate a reviewable AWS Support request. It must not fabricate compliance, volumes, customer relationships, or approval status.

5. Non-Functional Requirements

Category	Requirement	Target
Security	AES-256 authenticated encryption for stored AWS secrets	Mandatory
API security	API keys stored only as cryptographic hashes	Mandatory
Latency	Valid ingestion should normally return within 100 ms server-side	Target
Durability	Acknowledged jobs must not silently disappear	Mandatory
Reliability	Exponential backoff with jitter for retryable failures	Mandatory
Scalability	Stateless API and horizontally scalable workers	Mandatory
Observability	Correlation IDs, metrics, logs and alerts	Mandatory
Privacy	Never log raw secrets or authorization headers	Mandatory
Availability	Application target of 99.9% monthly availability	Target

6. Database Schema

The following Drizzle ORM baseline is the requested core schema.

```
import {
  pgTable, pgEnum, uuid, text, varchar, timestamp,
  boolean, integer, jsonb, uniqueIndex, index
} from "drizzle-orm/pg-core";

export const environmentEnum = pgEnum("environment", ["staging", "production"]);
export const domainStatusEnum = pgEnum("domain_status", ["pending", "verified", "failed"]);
export const emailStatusEnum = pgEnum("email_status", [
  "queued", "sending", "sent", "delivered",
  "bounced", "complained", "failed", "suppressed"
]);

export const users = pgTable("users", {
  id: uuid("id").primaryKey(),
  email: varchar("email", { length: 320 }).notNull().unique(),
  displayName: varchar("display_name", { length: 120 }),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
  updatedAt: timestamp("updated_at", { withTimezone: true }).defaultNow().notNull(),
});

export const awsCredentials = pgTable("aws_credentials", {
  id: uuid("id").defaultRandom().primaryKey(),
  userId: uuid("user_id").notNull().references(() => users.id, { onDelete: "cascade" }),
  environment: environmentEnum("environment").notNull(),
  accessKeyId: varchar("access_key_id", { length: 128 }).notNull(),
  secretAccessKeyCiphertext: text("secret_access_key_ciphertext").notNull(),
  awsRegion: varchar("aws_region", { length: 32 }).notNull(),
  sesMaxSendRate: integer("ses_max_send_rate"),
  active: boolean("active").default(true).notNull(),
  lastValidatedAt: timestamp("last_validated_at", { withTimezone: true }),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
}, (t) => ({
  userEnv: uniqueIndex("aws_credentials_user_env").on(t.userId, t.environment),
}));

export const verifiedDomains = pgTable("verified_domains", {
  id: uuid("id").defaultRandom().primaryKey(),
  userId: uuid("user_id").notNull().references(() => users.id, { onDelete: "cascade" }),
  domain: varchar("domain", { length: 253 }).notNull(),
  environment: environmentEnum("environment").notNull(),
  status: domainStatusEnum("status").default("pending").notNull(),
  sesIdentityArn: text("ses_identity_arn"),
  dkimTokens: jsonb("dkim_tokens"),
  spfRecord: text("spf_record"),
  dmarcRecord: text("dmarc_record"),
  lastCheckedAt: timestamp("last_checked_at", { withTimezone: true }),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
}, (t) => ({
  userDomainEnv: uniqueIndex("verified_domains_user_domain_env")
    .on(t.userId, t.domain, t.environment),
}));
```

```
export const emailTemplates = pgTable("email_templates", {
  id: uuid("id").defaultRandom().primaryKey(),
  userId: uuid("user_id").notNull().references(() => users.id, { onDelete: "cascade" }),
  name: varchar("name", { length: 120 }).notNull(),
  subject: varchar("subject", { length: 998 }).notNull(),
  htmlBody: text("html_body").notNull(),
  textBody: text("text_body"),
  variables: jsonb("variables"),
  version: integer("version").default(1).notNull(),
  active: boolean("active").default(true).notNull(),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
  updatedAt: timestamp("updated_at", { withTimezone: true }).defaultNow().notNull(),
}, (t) => ({
  userName: uniqueIndex("email_templates_user_name").on(t.userId, t.name),
}));

export const emailLogs = pgTable("email_logs", {
  id: uuid("id").defaultRandom().primaryKey(),
  userId: uuid("user_id").notNull().references(() => users.id, { onDelete: "cascade" }),
  environment: environmentEnum("environment").notNull(),
  status: emailStatusEnum("status").default("queued").notNull(),
  idempotencyKey: varchar("idempotency_key", { length: 255 }),
  messageId: varchar("message_id", { length: 255 }),
  sesRequestId: varchar("ses_request_id", { length: 255 }),
  fromAddress: varchar("from_address", { length: 320 }).notNull(),
  recipients: jsonb("recipients").notNull(),
  subject: varchar("subject", { length: 998 }).notNull(),
  templateId: uuid("template_id").references(() => emailTemplates.id, { onDelete: "set null" }),
  tags: jsonb("tags"),
  errorCode: varchar("error_code", { length: 120 }),
  errorMessage: text("error_message"),
  providerEvent: jsonb("provider_event"),
  queuedAt: timestamp("queued_at", { withTimezone: true }).defaultNow().notNull(),
  sentAt: timestamp("sent_at", { withTimezone: true }),
  completedAt: timestamp("completed_at", { withTimezone: true }),
}, (t) => ({
  userCreated: index("email_logs_user_queued").on(t.userId, t.queuedAt),
  message: index("email_logs_message").on(t.messageId),
  idem: uniqueIndex("email_logs_user_idempotency").on(t.userId, t.idempotencyKey),
}));

export const suppressionList = pgTable("suppression_list", {
  id: uuid("id").defaultRandom().primaryKey(),
  userId: uuid("user_id").notNull().references(() => users.id, { onDelete: "cascade" }),
  email: varchar("email", { length: 320 }).notNull(),
  reason: varchar("reason", { length: 40 }).notNull(),
  sourceEventId: varchar("source_event_id", { length: 255 }),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
  active: boolean("active").default(true).notNull(),
}, (t) => ({
  userEmail: uniqueIndex("suppression_user_email").on(t.userId, t.email),
}));

export const apiKeys = pgTable("api_keys", {
  id: uuid("id").defaultRandom().primaryKey(),
  userId: uuid("user_id").notNull().references(() => users.id, { onDelete: "cascade" }),
  name: varchar("name", { length: 120 }).notNull(),
  keyPrefix: varchar("key_prefix", { length: 16 }).notNull(),
  keyHash: text("key_hash").notNull(),
  environment: environmentEnum("environment").notNull(),
  lastUsedAt: timestamp("last_used_at", { withTimezone: true }),
  expiresAt: timestamp("expires_at", { withTimezone: true }),
  revokedAt: timestamp("revoked_at", { withTimezone: true }),
  createdAt: timestamp("created_at", { withTimezone: true }).defaultNow().notNull(),
}, (t) => ({
  hash: uniqueIndex("api_keys_hash").on(t.keyHash),
  userEnv: index("api_keys_user_env").on(t.userId, t.environment),
}));
```

Recommended additional production tables

Add `sns_events` with unique SNS `MessageId`, `audit_logs`, and optionally `usage_counters` and a provider-neutral event table.

7. API Specification

Authentication

- Dashboard: Supabase Auth.
- Send API: `Authorization: Bearer <API_KEY>`.
- API keys are environment-scoped and tenant-scoped.

POST `/api/v1/send`

```
Authorization: Bearer sk_live_...
Content-Type: application/json
Idempotency-Key: 4a9b...
```

```
{
  "from": "noreply@example.com",
  "to": ["alice@example.com", "bob@example.com"],
  "subject": "Welcome {{first_name}}",
  "html": "<h1>Hello {{first_name}}</h1>",
  "text": "Hello {{first_name}}",
  "templateId": "uuid-optional",
  "variables": {"first_name": "Alice"},
  "tags": {"campaign": "onboarding"}
}
```

202 Accepted

```
{
  "id": "email-log-uuid",
  "status": "queued",
  "message": "Accepted for asynchronous delivery"
}
```

Errors:

- `400` invalid request
- `401` invalid/revoked API key
- `403` sender/domain/environment not authorized
- `409` idempotency conflict
- `429` application rate limit exceeded
- `503` queue unavailable

POST `/api/v1/webhooks/aws-sns`

```
Content-Type: application/json
```

The body may be an SNS `SubscriptionConfirmation`, `Notification`, or `Unsubscribe` message.

Processing:

1. Verify SNS signature.
2. Validate `TopicArn`.
3. Use SNS `MessageId` as idempotency key.
4. Parse SES notification.
5. Update `email_logs`.
6. Upsert suppression records for applicable events.
7. Return 2xx after durable processing.

GET `/api/v1/domains/verify`

```
GET /api/v1/domains/verify?domain=example.com&environment=production
```

```
{
  "domain": "example.com",
  "environment": "production",
  "status": "verified",
  "checks": {
    "sesIdentity": "verified",
    "spf": {"status": "pass"},
    "dkim": {"status": "pass"},
    "dmarc": {"status": "pass"}
  },
  "checkedAt": "2026-09-26T16:00:00Z"
}
```

Error envelope

```
{
  "error": {
    "code": "DOMAIN_NOT_VERIFIED",
    "message": "The sender domain is not verified for this environment.",
    "requestId": "req_01..."
```

```
  }
}
```

8. Queueing, Retry & Delivery Semantics

1. Validate request.
2. Check suppression.
3. Resolve and render template.
4. Create `email_logs` record in `queued`.
5. Durably enqueue BullMQ job.
6. Return `202`.
7. Worker obtains rate-limit token.
8. Worker loads customer credential.
9. Worker submits to SES.
10. Persist SES response.
11. SNS later updates delivery/bounce/complaint state.

Condition	Behavior
Validation error	Reject immediately
Suppressed recipient	Do not call SES
SES throttling	Exponential retry + jitter
Network/5xx	Retry within bounded attempts
Permanent SES rejection	Fail without infinite retries
Worker crash	Job remains recoverable
Duplicate job	Idempotency/state checks
Duplicate webhook	SNS MessageId deduplication

Suggested policy: 5–8 attempts, exponential delay with jitter, capped around 15–30 minutes, followed by a terminal failure/dead-letter state.

9. Security & Secrets Management

- Encrypt AWS secret keys with AES-256-GCM or equivalent authenticated encryption.
- Prefer AWS KMS or another managed secret mechanism for the master key.
- Generate API keys using a cryptographically secure random generator.
- Store only API-key hashes.
- Display an API key secret once.
- Use least-privilege IAM.
- Enforce tenant isolation at application and, where appropriate, PostgreSQL RLS layers.
- Validate SNS signatures and trusted topic ARNs.
- Use TLS everywhere.
- Use secure cookies.
- Implement CSRF controls for cookie-authenticated mutations.
- Validate and constrain template variables.
- Rate-limit authentication, API-key creation, domain verification and webhook abuse.
- Never log credentials, bearer tokens, full message bodies, or unnecessary PII.

10. DNS & Domain Verification

Onboarding sequence:

1. Create SES identity.
2. Retrieve SES verification/DKIM information.
3. Publish DNS records.
4. Poll SES verification.
5. Validate SPF.
6. Validate DMARC.
7. Test sending identity.

The dashboard should detect missing records, malformed values, duplicate SPF records and incorrect DKIM tokens where technically observable.

Check	Expected state
SES identity	Verified
SPF	Valid/aligned record
DKIM	Required tokens resolve
DMARC	TXT at `_dmarc.domain`

11. Webhook & Suppression Processing

SES events flow through SNS to the public webhook.

The receiver must verify the SNS message, persist a bounded/redacted event representation, map provider identifiers to message logs, and update recipient state.

Hard bounces and complaints should normally activate suppression. Transient/soft bounce behavior should be policy-driven.

Suppression must be checked:

- before queue insertion
- immediately before provider submission

Manual unsuppression, if implemented, must be explicit and auditable.

12. Dashboard & Analytics

Screen	Required capabilities
Overview	1/7/30-day metrics, delivery rate, bounce/complaint counts, queue depth
Activity	Search by recipient, message ID, subject, template, status, date
Domains	Verification state, SPF/DKIM/DMARC checklist, region
Templates	Editor, versions, variables, preview, test send
API Keys	Create, revoke, rotate, environment scope
AWS	Credential status, region, quota snapshot, sandbox state
Suppressions	Search, reason, source event, active state
System Health	Queue depth, retries, failures, webhook lag, worker heartbeat

13. AWS Sandbox Exit Assistant

The assistant gathers use-case, website, expected volume, sending frequency, recipient acquisition process, bounce/complaint controls, suppression, unsubscribe handling, domain authentication, AWS region and requested quota.

The generated request must be clearly marked as a draft for human review. It must never fabricate compliance evidence or guarantee approval.

14. Error Model & Observability

Code	Meaning
`INVALID_REQUEST`	Schema or validation failure
`UNAUTHORIZED`	Invalid API key/session
`DOMAIN_NOT_VERIFIED`	Sender identity unavailable
`RECIPIENT_SUPPRESSED`	Recipient is suppressed
`SES_THROTTLED`	Provider quota/rate exceeded
`SES_REJECTED`	Permanent provider rejection
`QUEUE_UNAVAILABLE`	Could not durably enqueue
`INTERNAL_ERROR`	Unexpected platform failure

Every request receives a correlation/request ID. Structured logs should include tenant, environment, request ID, job ID, status transition and latency while excluding sensitive data.

15. Deployment & Infrastructure

Component	Deployment	Responsibility
Next.js 15	Vercel	Dashboard, API, webhooks
PostgreSQL	Supabase	Durable state
Redis	Upstash	Queue backend/rate limiting
BullMQ workers	Persistent worker runtime	SES delivery
Amazon SES	Customer AWS account	Email submission
AWS SNS	Customer AWS account	Event transport
Customer DNS	DNS provider	SPF/DKIM/DMARC

Important: long-lived BullMQ workers should not be treated as ordinary Vercel serverless request handlers. Use a worker runtime compatible with persistent/background processing.

16. Testing & Acceptance Criteria

Area	Acceptance criterion
Auth	Tenant isolation prevents cross-user access
Credentials	Secret never returned after creation
Domains	Correct SES/DNS state is displayed
Send API	Valid request returns `202` after durable enqueue
Suppression	Suppressed recipients never reach SES
Queue	Worker failure does not silently lose accepted jobs
Retries	Retryable errors back off; permanent errors terminate
Webhooks	Signature validation and idempotency work
Templates	Variables render deterministically
Analytics	State transitions appear in dashboard
Security	No secret leakage; revoked key cannot send
Performance	Server-side ingestion meets `<100 ms` target under normal load
Recovery	Documented recovery does not silently lose messages

Recommended test suites

- Unit tests
- PostgreSQL/Redis integration tests
- API contract tests
- SES sandbox integration tests
- Load tests

- Security/tenant-isolation tests
- Webhook replay tests
- Worker crash/chaos tests

17. Operational Runbook

SES throttling spike

Inspect queue depth, configured `MaxSendRate`, quota changes and worker concurrency. Reduce concurrency before increasing quota.

Bounce/complaint spike

Inspect affected domains, suppression processing, recipient acquisition and recent templates. Pause risky sending if required.

Queue backlog

Inspect worker heartbeat, Redis health, retries and provider errors.

Webhook outage

Verify endpoint availability, SNS subscription state, signature validation and application health. Reconcile events after recovery where possible.

Credential failure

Mark the credential invalid, stop affected jobs, ask the user to rotate/revalidate keys, and never expose the secret in logs.

Database incident

Preserve queue durability, avoid duplicate sends, restore from managed backups and reconcile state using idempotent identifiers.

18. Risks, Constraints & Future Extensions

Risk	Mitigation
AWS quotas	Surface quotas and provide quota workflow
Reputation	Suppression, authentication, monitoring
Duplicate delivery	Idempotency and conservative retry classification
Redis outage	Durable queue configuration and backpressure
Credential compromise	Least privilege, encryption, rotation
DNS propagation	Clear last-check state and bounded polling
Provider dependency	Internal provider abstraction
Cost growth	Usage metering and future billing

Future extensions

- Node.js/Python/Go SDKs
- Scheduled sends
- Batch APIs
- Webhook replay/reconciliation
- Multi-provider failover
- Usage metering and billing
- Teams/RBAC
- Provider-neutral email abstraction
- Advanced deliverability analytics

Appendix A — Reference Send Lifecycle

```
1. Client authenticates with API key.
2. API validates schema, sender and recipients.
3. API checks suppression and idempotency.
4. Template is rendered.
5. Email log is created as QUEUED.
6. BullMQ job is durably added to Redis.
7. API returns 202.
8. Worker acquires distributed SES rate-limit token.
9. Worker loads/decrypts AWS credential.
10. Worker calls SES.
11. Provider response updates the log.
12. SNS events update DELIVERED/BOUNCED/COMPLAINED.
13. Bounce/complaint recipients enter suppression.
14. Dashboard reflects resulting state.
```

Appendix B — Engineering Definition of Done

- FR-1 through FR-8 implemented and tested.
- Mandatory NFRs have implementation and test evidence.
- Database migrations are versioned and reproducible.
- Secret management passes security review.
- Queue retry/dead-letter behavior is documented and tested.
- SNS signature verification and idempotency are tested.
- Tenant isolation has automated negative tests.
- Monitoring exists for queue backlog, throttling, webhook failures, error rate and credential failures.
- Rollback and incident procedures are documented.

End of SRS